

Databases

Relational Data Model

A **relation** is a **table** in a database, referring to the logical structure of the table not how it is physically stored.

Rows in a table are called **tuples**, **columns** are called **attributes**.

The **degree** of a **relation** is the number of **columns**.

The **cardinality** of a **relation** is the number of **rows**.

The **domain** of an **attribute** is the **set of allowed values** for that attribute.

A **key** is a set of **attributes** which uniquely identifies a **tuple**.

A **candidate key** is a **minimal set** of attributes which is a **key**. One is chosen to be the **primary key**.

A **key** consisting of multiple attributes is a **composite key**.

A **foreign key** is an **attribute** that references the **primary key** of another relation.

Integrity Constraints:

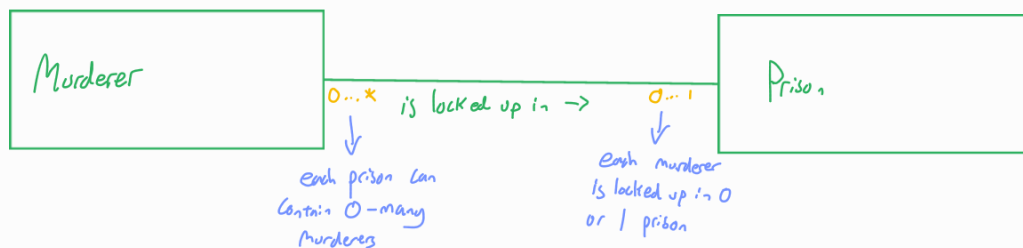
- **Domain constraints** - attribute values must lie within the **domain** of the attribute.
- **Entity integrity** - **primary key** attributes cannot be Null
- **Referential integrity** - **foreign key** attributes must match another relation's primary key or be Null

Views:

A **view** is a **virtual relation** (i.e. not physically stored in DB) computed on request by a user, containing different information depending on the user, and may include **derived attributes**, e.g. date of birth is stored in DB, but the age is a derived attribute displayed to the user in a view.

Entity Relationship Model

Entity-Relationship modelling shows **entities** with **attributes**, as well as **relationships** between **entities** and their **constraints**.



Normalization

Functional dependencies - A **set of attributes A** is **functionally dependent** on another **set of attributes B** if each value of **A** is entirely determined by a **unique value of B**.

E.g. in a table consisting of attributes **id**, **firstname**, **surname**, **college**, **collegeAddress**:

- **collegeAddress** is functionally dependent on **college** → for a given college the collegeAddress is always the same
- **surname** is **NOT** functionally dependent on **firstname** → for a given firstname the surname may not always be the same, so we cannot determine the surname from just the firstname.

A **full functional dependency** is where **A** is functionally dependent on **B** but **not** on any proper subset of **B**.

A **partial functional dependency** is where **A** is functionally dependent on **B** but also remains functionally dependent on a proper subset of **B**.

If **C** is dependent on **B** and **B** is dependent on **A**, then **C** is **transitively dependent** on **A**.

First Normal Form (1NF) - A table is in **1NF** if every cell has one value, i.e. there are no repeating groups of attribute

- fix by creating separate relation for the repeated data.

Second Normal Form (2NF) - A table is in **2NF** if it is in **1NF** and there are **no partial functional dependencies**

- fix by moving partial dependencies into a new relation with the attribute they were dependent on as the primary key.

Third Normal Form (3NF) - A table is in 3NF if it is in 2NF and has no transitive dependencies
 - fix by moving transitively dependent attributes into a new relation, with their direct dependent as primary key.

SQL

SELECT Statement:

SELECT id, firstName, lastName, Salary/12 AS monthlySalary
FROM employees
WHERE Salary/12 BETWEEN 500 AND 1000

derived attribute, given the alias 'monthlySalary'

Cannot use aliases in WHERE clause
 → the where clause is processed before the SELECT clause

Only select distinct values.

SELECT DISTINCT firstName, lastName, Salary, Position
FROM employees
WHERE firstName LIKE 'J...' **AND** lastName LIKE '%h'
ORDER BY Salary DESC, lastName ASC

lastName must start with J and be exactly 4 characters total
 lastName must end in h
 → % is the wildcard for LIKE statements.

If multiple have the same salary, order those by
 lastName in ascending order.

SELECT position, COUNT(position) AS Count
FROM employees
GROUP BY position

Aggregate function - returns how many non-NULL
 values are in a column

→ other aggregate fncs include SUM, AVG, MIN, MAX

introduces a new row in the
 returned table for each position
 → only use with aggregate
 functions

Example return data:

Position	Count
Bossman	1
Assistant	5
Sum	28

JOINS:

Table 1

id1	attribute1
0	A
1	B
2	D

Table 2

id2	attribute2
798	R
799	B
800	G

SELECT *

FROM Table1 **INNER JOIN** Table2
ON attribute1 = attribute2

Only return all matching combinations

id1	attribute1	id2	attribute2
1	B	799	B

SELECT *

FROM Table1 **LEFT JOIN** Table2
ON attribute1 = attribute2

Return all matching combinations, plus all leftover
 rows from left table

id1	attribute1	id2	attribute2
1	B	799	B
0	A	NULL	NULL
2	D	NULL	NULL

SELECT *

FROM Table1 RIGHT JOIN Table2

ON attribute1 = attribute2

Returns all matching combinations, plus all leftover rows from right table

id1	attribute1	id2	attribute2
1	B	799	B
NULL	NULL	798	R
NULL	NULL	800	G

SELECT *

FROM Table1 FULL OUTER JOIN Table2

ON attribute1 = attribute2

Returns all matching combinations, plus all leftover rows from both tables.

id1	attribute1	id2	attribute2
1	B	799	B
0	A	NULL	NULL
2	D	NULL	NULL
NULL	NULL	798	R
NULL	NULL	800	G

Subqueries:

SELECT firstName, lastName, position, salary

FROM employees

WHERE salary > ALL (SELECT salary
FROM employees
WHERE position = 'cleaner')

SELECT firstName, lastName, position, salary

FROM employees

WHERE salary > (SELECT MAX(salary)
FROM employees
WHERE position = 'cleaner')

both of these logically do the same thing.

INSERT statement:

INSERT INTO employees(id, firstName, lastName, position, salary)
VALUES (365, Jonathan, Pigylesunde, Secretary, 17000)

Inserts a new record into the table

UPDATE statement:

UPDATE employees
SET salary = 1.5 * salary
WHERE position = 'Bossman'

Updates record(s) in a table
WHERE clause can be omitted to update all

DELETE statement:

DELETE FROM employees
WHERE id = 7

Deletes record(s) from a table
WHERE clause can be omitted to delete all

DROP TABLE statement:

DROP TABLE shiteTable

Deletes a table

CREATE TABLE statement:

CREATE TABLE users (

Creates a new table

Username VARCHAR(32) NOT NULL UNIQUE,
 Email VARCHAR(64),
 Password VARCHAR(64) NOT NULL,
 EmployeeID INTEGER,
 PRIMARY KEY (Username),
 FOREIGN KEY (EmployeeID) REFERENCES employees(id)
 ON DELETE SET NULL
 ON UPDATE CASCADE

Datatypes:
 - CHAR(n)
 - VARCHAR(n)
 - BIT
 - INTEGER
 - NUMERIC

)

ALTER TABLE Statement:

ALTER TABLE users
 ADD phone VARCHAR(20)

ALTER TABLE users
 DROP email CASCADE

Add and remove attributes from a relation.

Relational Calculus and Algebra

Relational Calculus is basically rewriting SQL statements using Set notation.

$\{ \text{employee} \mid \text{employees}(\text{employee}) \wedge \text{employee.position} = \text{'Bossman'} \}$

is the same as

SELECT *
 FROM employees
 WHERE position = 'Bossman'

Relational algebra is another bullshit way of writing SQL with mathematical symbols.

- Selection $\sigma_{\text{condition}}(\text{Relation})$ - Equivalent to WHERE clause in SQL:

$\sigma_{\text{FirstName} = \text{'John'}}(\text{employees})$

is equivalent to

SELECT *
 FROM employees
 WHERE firstName = 'John'

- Projection $\pi_{\text{attr}_1, \dots}(\text{Relation})$ - Equivalent to SELECT DISTINCT in SQL - automatically removes duplicates.

$\pi_{\text{position, salary}}(\text{employees})$

is equivalent to

SELECT position, salary
 FROM employees

- Union $\cup$ and Intersection $\cap$ - as the basic set operations, duplicates are removed automatically.
 - Set Difference $\text{Relation}_1 - \text{Relation}_2$ - Selects tuples in Relation₁ but not in Relation₂

To use these operations, relations must have the same number of attributes and equivalent attributes in both relations must have the same domain - name doesn't matter

- Cartesian Product $\text{Relation}_1 \times \text{Relation}_2$ - returns all possible combinations of concatenations of tuples in the two relations

- Rename $\rho_{\text{newName}}(\text{oldName})$ - used to give a name to an intermediate expression to make it nicer to reuse.

- Joins - used to express the different joins in SQL

$A \bowtie_{\text{attr}_1 = \text{attr}_2} B$ A INNER JOIN B ON attr₁ = attr₂

$A \ltimes_{\text{attr}_1 = \text{attr}_2} B$ A LEFT JOIN B ON attr₁ = attr₂

$A \bowtie_{attr_1 = attr_2} B$

$A \text{ RIGHT JOIN } B \text{ ON } attr_1 = attr_2$

$A \bowtie_{attr_1 = attr_2} B$

$A \text{ FULL OUTER JOIN } B \text{ ON } attr_1 = attr_2$